EN3160 Assignment 2 on Fitting and Alignment

230680M - W.M.H. Wanigasundara

GitHub: https://github.com/HasinduWanigasundara/Fitting_and_Alignment

Problem 1

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import maximum_filter

im = cv2.imread("images/the_berry_farms_sunflower_field.jpeg",
                cv2.IMREAD_REDUCED_COLOR_4)
f = im.astype(np.float32) / 255.0
yellowness = np.clip((f[..., 2] + f[..., 1]) / 2 - f[..., 0], 0, 1)  # petals
red_minus_green = f[..., 2] - f[..., 1]   # brown centre > 0, leaves < 0
gray = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY).astype(np.float32) / 255.0

# Scale-normalized LoG: dark blobs (sunflower centres) give positive peaks
sigmas = np.geomspace(1.0, 16.0, 25)
responses = np.stack([
    s ** 2 * cv2.Laplacian(cv2.GaussianBlur(gray, (0, 0), s), cv2.CV_32F, ksize=3)
    for s in sigmas
])

# Scale-space maxima; a maximum at the first or last sigma is not a true extremum
threshold = 0.1 * responses.max()
local_max = maximum_filter(responses, size=(3, 7, 7), mode="constant", cval=-np.inf)
extrema = (responses == local_max) & (responses > threshold)
extrema[[0, -1]] = False

h, w = gray.shape
yy, xx = np.mgrid[:h, :w]
candidates = []

for i, y, x in zip(*np.where(extrema)):
    r = np.sqrt(2) * sigmas[i]
    if x < r or y < r or x + r >= w or y + r >= h:  # cut by the image border
        continue
    # Keep dark blobs with a ring of yellow petals and a brownish centre
    d = np.hypot(xx - x, yy - y)
    if (yellowness[(d >= r) & (d <= 2 * r)].mean() > 0.25
            and red_minus_green[d <= max(0.7 * r, 1)].mean() > 0):
        candidates.append((responses[i, y, x], int(x), int(y), r, sigmas[i]))

# Remove overlapping detections, keeping the stronger response
circles = []
for c in sorted(candidates, reverse=True):
    if all(np.hypot(c[1] - e[1], c[2] - e[2]) >= 0.9 * (c[3] + e[3]) for e in circles):
        circles.append(c)

largest = sorted(circles, key=lambda c: c[3], reverse=True)[:10]

# All circles in green, the five largest in red
result = cv2.cvtColor(im, cv2.COLOR_BGR2RGB)
for _, x, y, r, _ in circles:
    cv2.circle(result, (x, y), int(round(r)), (0, 255, 0), 1)
for _, x, y, r, _ in largest[:5]:
    cv2.circle(result, (x, y), int(round(r)), (255, 0, 0), 2)

plt.figure(figsize=(12, 10))
plt.imshow(result)
```

```
plt.title(f"Sunflower Detection using LoG ({len(circles)} detections, five largest in red)")
plt.axis("off")
plt.show()

print(f"{'X':>6} {'Y':>6} {'Radius':>8} {'Sigma':>7} {'Response':>9}")
for score, x, y, r, sigma in largest:
    print(f"{x:6d} {y:6d} {r:8.2f} {sigma:7.2f} {score:9.4f}")
print(f"\nSigma range: {sigmas[0]:.2f} to {sigmas[-1]:.2f} ({len(sigmas)} log-spaced scales)")
print(f"Response threshold: {threshold:.4f}, total detections: {len(circles)}")
```

Sunflower Detection using LoG (101 detections, five largest in red)



X	Y	Radius	Sigma	Response
282	338	10.08	7.13	1.3955
106	255	8.00	5.66	1.4391
178	261	8.00	5.66	1.4008
179	327	8.00	5.66	1.3608
189	240	6.35	4.49	1.4428
161	290	6.35	4.49	1.4374
63	243	6.35	4.49	1.4134
70	286	6.35	4.49	1.3844
274	258	6.35	4.49	1.3702
19	251	6.35	4.49	1.1588

```
Sigma range: 1.00 to 16.00 (25 log-spaced scales)
Response threshold: 0.1540, total detections: 101
```

Method. The scale-normalized LoG $\sigma^2 \nabla^2 G_\sigma * I$ is computed on the grayscale image for 25 log-spaced scales $\sigma \in [1, 16]$. A sunflower's dark centre within bright petals is a strong *dark* blob, so positive scale-space maxima ($3 \times 7 \times 7$ neighbourhood, above $0.1\times$ the global maximum) are detections, with $r = \sqrt{2}\,\sigma$.

A plain LoG also fires on the tree line and leaf gaps, so a detection is kept only if the ring $r \le d \le 2r$ is yellow and its centre brownish ($R > G$; leaves have $G > R$). Maxima at the end scales, blobs cut by the border and overlaps are removed.

The five largest circles have $\sigma \approx 4.5$–7.1 ($r = 6.3$–10.1 px), well inside the range: the large foreground sunflowers. Distant flowers give smaller σ (perspective).

Problem 2

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
from scipy.optimize import minimize

np.random.seed(0)

# Noisy point set: first half of X is the circle, second half the line
N, half_n = 100, 50
r_gt, x0_gt, y0_gt = 10, 2, 3
t = np.random.uniform(0, 2 * np.pi, half_n)
n = r_gt / 16 * np.random.randn(half_n)
X_circ = np.column_stack((x0_gt + (r_gt + n) * np.cos(t), y0_gt + (r_gt + n) * np.sin(t)))
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
X_line = np.column_stack((x, m * x + b + 1.0 * np.random.randn(half_n)))
X = np.vstack((X_circ, X_line))


# Line ax + by = d with unit normal [a, b] and d >= 0
def line_model(normal, point):
    normal = normal / np.linalg.norm(normal)
    d = normal @ point
    return (*(-normal), -d) if d < 0 else (*normal, d)


def line_from_points(p):
    direction = p[1] - p[0]
    if np.linalg.norm(direction) < 1e-10:
        return None
    return line_model(np.array([-direction[1], direction[0]]), p[0])


def fit_line_tls(p):
    center = p.mean(axis=0)
    return line_model(np.linalg.svd(p - center)[2][-1], center)


def line_distances(points, model):
    return np.abs(points @ model[:2] - model[2])


def circle_from_three_points(p):
    A = 2 * np.array([p[1] - p[0], p[2] - p[0]])
    if abs(np.linalg.det(A)) < 1e-6:
        return None
    center = np.linalg.solve(A, [p[1] @ p[1] - p[0] @ p[0], p[2] @ p[2] - p[0] @ p[0]])
    return (*center, np.linalg.norm(p[0] - center))


def circle_errors(points, model):
    return np.abs(np.hypot(*(points - model[:2]).T) - model[2])


def fit_circle(points, initial):
    # Least-squares radial error, minimized with scipy
    cost = lambda params: np.sum(circle_errors(points, params) ** 2)
    return minimize(cost, initial, method="L-BFGS-B",
                    bounds=[(None, None), (None, None), (1e-6, None)]).x


def ransac(points, sample_size, fit, errors, threshold, min_inliers, iterations):
    # Returns the model of the best sample, the sample indices and its inliers
    best_count, best_error, best = -1, np.inf, None
    for _ in range(iterations):
        sample = np.random.choice(len(points), sample_size, replace=False)
        model = fit(points[sample])
```

```
        if model is None:
            continue
        e = errors(points, model)
        mask = e < threshold
        count, error = mask.sum(), np.sum(e[mask] ** 2)
        if count >= min_inliers and (count, -error) > (best_count, -best_error):
            best_count, best_error, best = count, error, (model, sample, mask)
    if best is None:
        raise RuntimeError("No model reached the minimum consensus size.")
    return best


def ransac_iterations(w, s, p=0.99):
    return int(np.ceil(np.log(1 - p) / np.log(1 - w ** s)))


# (a) Line: perpendicular noise is 1/sqrt(2) = 0.71, so 1.5 is about 2.1 sigma;
# at least 40 of the 50 line points
line_threshold, line_min_inliers = 1.5, 40
line_sample_model, line_sample, line_inliers = ransac(
    X, 2, line_from_points, line_distances, line_threshold, line_min_inliers, 3000)

# Refine with total least squares on the consensus until the inliers stop changing
for _ in range(5):
    line_fit = fit_line_tls(X[line_inliers])
    new_inliers = line_distances(X, line_fit) < line_threshold
    if np.array_equal(new_inliers, line_inliers):
        break
    line_inliers = new_inliers

# (b) Circle on the remnant: radial noise sigma = 0.625, so 1.2 is about 2 sigma
remaining_idx = np.where(~line_inliers)[0]
X_remaining = X[remaining_idx]
circle_threshold, circle_min_inliers = 1.2, 35
circle_sample_model, circle_sample, circle_mask = ransac(
    X_remaining, 3, circle_from_three_points, circle_errors,
    circle_threshold, circle_min_inliers, 5000)
circle_fit = fit_circle(X_remaining[circle_mask], circle_sample_model)
circle_inliers = circle_errors(X_remaining, circle_fit) < circle_threshold

print(f"Minimum iterations (p = 0.99): line {ransac_iterations(0.5, 2)}, "
      f"circle {ransac_iterations(0.8, 3)}")
print(f"\nLine: threshold {line_threshold}, minimum consensus {line_min_inliers}, "
      f"{line_inliers.sum()} inliers ({line_inliers[half_n:].sum()} line, "
      f"{line_inliers[:half_n].sum()} circle points)")
print("  sample line: {:.4f}x + {:.4f}y = {:.4f}".format(*line_sample_model))
print("  best fit:    {:.4f}x + {:.4f}y = {:.4f}, |[a, b]| = {:.4f}".format(
    *line_fit, np.hypot(*line_fit[:2])))
print(f"  slope {-line_fit[0] / line_fit[1]:.3f}, intercept "
      f"{line_fit[2] / line_fit[1]:.3f} (ground truth {m}, {b})")
print(f"\nCircle: threshold {circle_threshold}, minimum consensus {circle_min_inliers}, "
      f"{circle_inliers.sum()} of {len(X_remaining)} remaining points "
      f"({(remaining_idx[circle_inliers] >= half_n).sum()} line points)")
print("  sample circle: centre ({:.3f}, {:.3f}), r = {:.3f}".format(*circle_sample_model))
print("  optimized:     centre ({:.3f}, {:.3f}), r = {:.3f}".format(*circle_fit))
print(f"  ground truth:  centre ({x0_gt}, {y0_gt}), r = {r_gt}")

# (c) Plot
fig, ax = plt.subplots(figsize=(10, 10))
ax.scatter(*X.T, s=18, color="steelblue", alpha=0.55, label="All points")
ax.scatter(*X[line_inliers].T, s=25, color="goldenrod", label="Line inliers")
ax.scatter(*X_remaining[circle_inliers].T, s=25, color="darkturquoise",
           label="Circle inliers")
ax.scatter(*X[line_sample].T, marker="x", s=100, color="red", linewidths=2,
           label="Best sample for line")
ax.scatter(*X_remaining[circle_sample].T, marker="x", s=100, color="black",
           linewidths=2, label="Best sample for circle")

x_plot = np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 400)
for (a_, b_, d_), style, color, label in [
        (line_sample_model, "--", "magenta", "Line from best RANSAC sample"),
```

```
        (line_fit, "-", "orange", "Best-fit line"),
        ((1, 1, 2), ":", "gray", "Ground-truth line")]:
    ax.plot(x_plot, (d_ - a_ * x_plot) / b_, linestyle=style, color=color,
            linewidth=1.8, label=label)

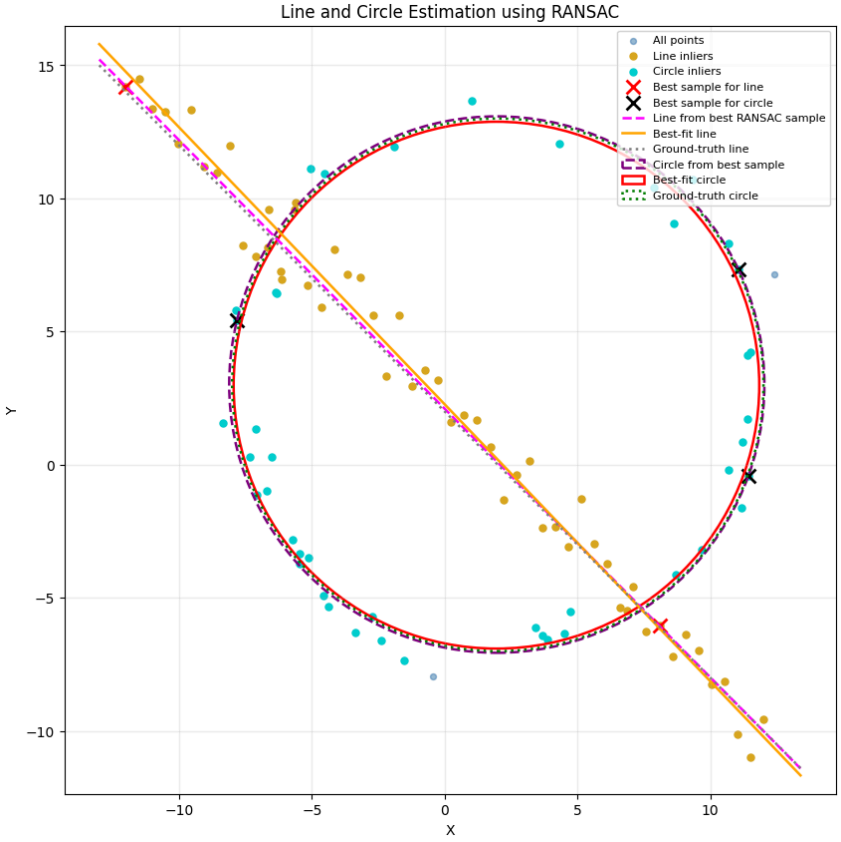
for (cx, cy, r), style, color, label in [
        (circle_sample_model, "--", "purple", "Circle from best sample"),
        (circle_fit, "-", "red", "Best-fit circle"),
        ((x0_gt, y0_gt, r_gt), ":", "green", "Ground-truth circle")]:
    ax.add_patch(Circle((cx, cy), r, fill=False, linestyle=style,
                        linewidth=1.8, color=color, label=label))

ax.set(xlabel="X", ylabel="Y", title="Line and Circle Estimation using RANSAC")
ax.set_aspect("equal", adjustable="datalim")
ax.grid(True, alpha=0.25)
ax.legend(fontsize=8, loc="upper right")
plt.show()
```

```
Minimum iterations (p = 0.99): line 17, circle 7

Line: threshold 1.5, minimum consensus 40, 54 inliers (50 line, 4 circle points)
  sample line: 0.7102x + 0.7040y = 1.4804
  best fit:    0.7210x + 0.6930y = 1.5724, |[a, b]| = 1.0000
  slope -1.040, intercept 2.269 (ground truth -1, 2)

Circle: threshold 1.2, minimum consensus 35, 44 of 46 remaining points (0 line points)
  sample circle: centre (1.962, 3.010), r = 10.072
  optimized:     centre (1.949, 2.988), r = 9.892
  ground truth:  centre (2, 3), r = 10
```



Parameter choices.

- **Line threshold $t = 1.5$:** RANSAC measures *perpendicular* distance, and the vertical noise ($\sigma = 1$) projects to $\sigma_\perp = 1/\sqrt{2} \approx 0.71$ for slope -1. So $t \approx 2.1\,\sigma_\perp$, keeping about 96% of line points (all 50 here). $t = 2.0$ took 8 circle points instead of 4; $t = 1.25$ left 3 line points for the circle.

- **Circle threshold $t = 1.2$:** the radial noise is $\sigma = r/16 = 0.625$, so $t \approx 1.9\sigma$ (about 95% of circle points).
- **Minimum consensus:** at least 40 of the 50 line points; at least 35 remaining points for the circle (it loses a few at the intersections).
- **Iterations:** $N = \log(1-p)/\log(1-w^s)$ with $p = 0.99$ gives 17 (line, $w = 0.5$, $s = 2$) and 7 (circle, $w \approx 0.8$, $s = 3$); 3000 and 5000 are ample.
- **Normal sign:** flipped so $d \geq 0$, so sample and best-fit lines share a sign; $\|[a, b]^\top\| = 1$.

The 4 circle points inside the line band tilt the fitted line slightly (slope -1.040). The optimized circle (centre $(1.95, 2.99)$, $r = 9.89$) is close to the truth ($(2, 3)$, $r = 10$). It has 44 inliers against 46 for the sample circle: least squares minimizes squared error, not inlier count, leaving two points just outside the band (errors 1.30, 1.34).

What will happen if we fit the circle first?

```
# (d) Experiment: fit the circle first on all points, then the line on the remnant
np.random.seed(1)

cf_sample_model, _, cf_mask = ransac(
    X, 3, circle_from_three_points, circle_errors,
    circle_threshold, circle_min_inliers, 5000)
cf_circle = fit_circle(X[cf_mask], cf_sample_model)
cf_circle_inliers = circle_errors(X, cf_circle) < circle_threshold

cf_remaining_idx = np.where(~cf_circle_inliers)[0]
_, _, cf_line_mask = ransac(X[cf_remaining_idx], 2, line_from_points,
                            line_distances, line_threshold, 0, 3000)
cf_line = fit_line_tls(X[cf_remaining_idx[cf_line_mask]])

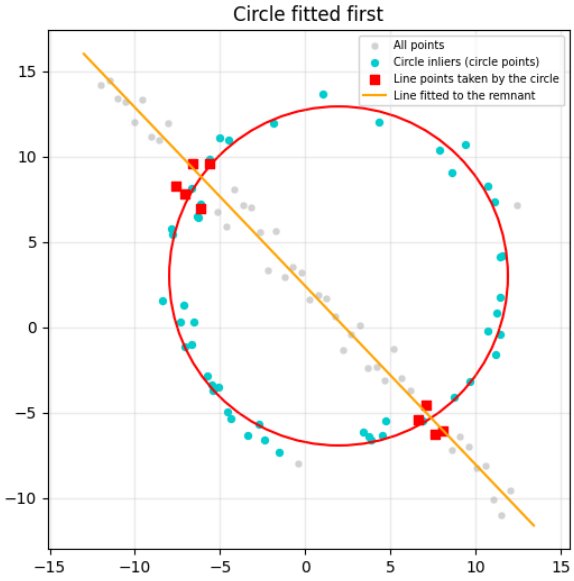
# Indices into X of each model's inliers (X[:half_n] circle, X[half_n:] line)
print(f"{'Order':<14}{'Circle inl.':>12}{'(line pts)':>12}"
      f"{'Line inl.':>11}{'(circle pts)':>14}{'Line pts lost':>15}")
for name, c_idx, l_idx in [
        ("Line first", remaining_idx[circle_inliers], np.where(line_inliers)[0]),
        ("Circle first", np.where(cf_circle_inliers)[0], cf_remaining_idx[cf_line_mask])]:
    print(f"{name:<14}{len(c_idx):>12}{(c_idx >= half_n).sum():>12}{len(l_idx):>11}"
          f"{(l_idx < half_n).sum():>14}{half_n - (l_idx >= half_n).sum():>15}")

print("\nCircle-first circle: centre ({:.3f}, {:.3f}), r = {:.3f}".format(*cf_circle))
print(f"Circle-first line slope/intercept: {-cf_line[0] / cf_line[1]:.3f}, "
      f"{cf_line[2] / cf_line[1]:.3f}")

taken = cf_circle_inliers & (np.arange(N) >= half_n)
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(*X.T, s=12, color="lightgray", label="All points")
ax.scatter(*X[cf_circle_inliers & ~taken].T, s=18, color="darkturquoise",
           label="Circle inliers (circle points)")
ax.scatter(*X[taken].T, s=40, color="red", marker="s",
           label="Line points taken by the circle")
ax.add_patch(Circle(cf_circle[:2], cf_circle[2], fill=False, color="red", linewidth=1.5))
ax.plot(x_plot, (cf_line[2] - cf_line[0] * x_plot) / cf_line[1], color="orange",
        label="Line fitted to the remnant")
ax.set_aspect("equal", adjustable="datalim")
ax.set_title("Circle fitted first")
ax.legend(fontsize=7, loc="upper right")
ax.grid(True, alpha=0.25)
plt.show()
```

```
Order          Circle inl.  (line pts)  Line inl.  (circle pts)  Line pts lost
Line first              44           0         54             4              0
Circle first            57           9         41             0              9
```

```
Circle-first circle: centre (1.943, 3.008), r = 9.930
Circle-first line slope/intercept: -1.047, 2.409
```



Answer. Fitting the circle first still finds the correct circle, since its 50 tightly packed points ($\sigma = 0.625$) give more support than any circle through line points. But its ± 1.2 band takes **9 line points** where the line crosses it, against 0 lost when the line goes first (which takes only 4 circle points). The line is then estimated from 41 points, with gaps at both intersections.

Line points are removed more often mainly because they are twice as dense: one every 0.69 units along the line against one every 1.26 around the circle. Their perpendicular noise ($\sigma_\perp \approx 0.71$) is also slightly larger. The circle, a 3-point model, also needs more samples ($N \propto 1/w^3$), and with the line points present, wrong circles through several line points are more likely. Fitting the line first therefore leaves a cleaner remnant for the circle.

Problem 3

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

INTERACTIVE = False  # True: click the corners again instead of using the saved ones
alpha = 0.98


def select_corners(image, title):
    # Click TL, TR, BR, BL; the window closes after the fourth click
    points = []

    def on_click(event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN:
            points.append((x, y))

    cv2.imshow(title, image)
    cv2.setMouseCallback(title, on_click)
    while len(points) < 4:
        cv2.waitKey(20)
    cv2.destroyWindow(title)
    return np.float32(points)


# (title, background, overlay, output file, corners clicked for the results)
pairs = [
    ("Naruto poster on billboard", "billboard", "naruto", "naruto_billboard.png",
```

```
     [[161, 144], [428, 137], [428, 541], [163, 536]]),
    ("Sri Lankan flag on smartboard", "classroom", "sri_lanka_flag",
     "sri_lankan_flag_smartboard.png", [[200, 120], [415, 122], [415, 254], [199, 253]]),
]

fig, ax = plt.subplots(2, 3, figsize=(15, 8))

for row, (title, bg_name, ov_name, out_name, corners) in enumerate(pairs):
    background = cv2.imread(f"images/{bg_name}.png")
    overlay = cv2.imread(f"images/{ov_name}.png")
    corners = select_corners(background, title) if INTERACTIVE else np.float32(corners)

    # Homography mapping the overlay's corners to the selected corners
    oh, ow = overlay.shape[:2]
    src = np.float32([[0, 0], [ow - 1, 0], [ow - 1, oh - 1], [0, oh - 1]])
    H = cv2.getPerspectiveTransform(src, corners)

    # Warp the overlay and the fraction of each pixel it covers, then blend;
    # partly covered edge pixels are mixed in proportion, so no dark outline
    size = background.shape[1::-1]
    warped = cv2.warpPerspective(overlay, H, size)
    cover = cv2.warpPerspective(np.ones((oh, ow), np.float32), H, size)[..., None]
    result = np.uint8(np.round(background * (1 - alpha * cover) + alpha * warped))

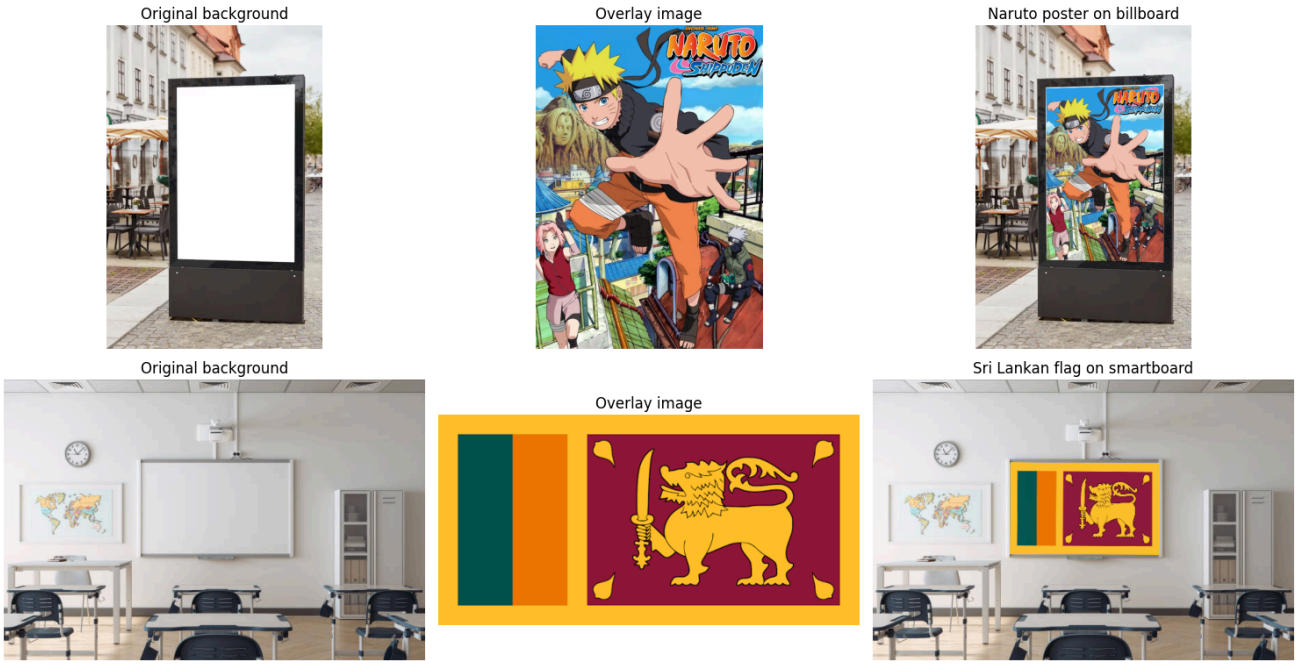
    cv2.imwrite(f"results/{out_name}", result)
    print(f"{title}\nCorners (TL, TR, BR, BL):\n{corners}\nHomography:\n{H}\n")

    for col, (im, label) in enumerate([(background, "Original background"),
                                       (overlay, "Overlay image"), (result, title)]):
        ax[row, col].imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB))
        ax[row, col].set_title(label)
        ax[row, col].axis("off")

plt.tight_layout()
fig.savefig("results/question3_comparison.png", dpi=100)
plt.show()
```

```
Naruto poster on billboard
Corners (TL, TR, BR, BL):
[[161. 144.]
 [428. 137.]
 [428. 541.]
 [163. 536.]]
Homography:
[[ 2.54501653e-01  2.26679907e-03  1.61000000e+02]
 [-1.10931967e-02  2.77926514e-01  1.44000000e+02]
 [-2.98262021e-05  5.29625952e-06  1.00000000e+00]]

Sri Lankan flag on smartboard
Corners (TL, TR, BR, BL):
[[200. 120.]
 [415. 122.]
 [415. 254.]
 [199. 253.]]
Homography:
[[ 3.45734540e-01 -6.09935332e-03  2.00000000e+02]
 [ 4.64107975e-03  4.18503821e-01  1.20000000e+02]
 [ 1.20615381e-05 -1.46972369e-05  1.00000000e+00]]
```



Rationale for the chosen pairs.

- **Naruto poster on an outdoor display:** previews a poster on a street display before printing; the display is a flat, clearly bounded plane.
- **Sri Lankan flag on a classroom smartboard:** the national flag shown for a school event such as National Day; the board is a slightly angled planar rectangle.

The overlay is almost opaque ($\alpha = 0.98$) as both surfaces are blank displays. The clicked corners are stored in the code, so no click window is needed to reproduce the output.

Problem 4

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

template = cv2.imread("images/AECC1.jpg")
template_gray = cv2.cvtColor(template, cv2.COLOR_BGR2GRAY)
th, tw = template_gray.shape

# Difference settings shared by all images so that results are comparable
blur_sigma = 2.0          # suppresses sensor noise and sub-pixel misalignment
diff_display_max = 80     # difference value shown as full brightness
change_threshold = 40     # gray-level difference treated as a real change
template_blur = cv2.GaussianBlur(template_gray.astype(np.float32), (0, 0), blur_sigma)

# (a) SIFT features of the template
sift = cv2.SIFT_create()
kp_template, des_template = sift.detectAndCompute(template_gray, None)
matcher = cv2.BFMatcher()
print("Template keypoints:", len(kp_template))

rgb = lambda im: cv2.cvtColor(im, cv2.COLOR_BGR2RGB)
small = lambda im: cv2.resize(im, None, fx=0.25, fy=0.25, interpolation=cv2.INTER_AREA)
rows, summary, match_panels = [], [], []

for name in ["AAAA1", "ADAD1", "AECC2", "AECC3", "AECC4"]:
    image = cv2.imread(f"images/{name}.jpg")
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    kp_image, des_image = sift.detectAndCompute(gray, None)
```

```
    # (b) Brute-force matching with Lowe's ratio test, then a RANSAC homography
    good = [m for m, n in matcher.knnMatch(des_image, des_template, k=2)
            if m.distance < 0.75 * n.distance]
    src = np.float32([kp_image[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp_template[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, inlier_mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    inliers = sorted([m for m, ok in zip(good, inlier_mask.ravel()) if ok],
                     key=lambda m: m.distance)

    # Thick lines, so the matches stay visible when the image is shrunk
    matches = cv2.drawMatches(image, kp_image, template, kp_template, inliers[:80], None,
                              matchColor=(0, 255, 0), matchesThickness=4,
                              flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    match_panels.append((rgb(small(matches)), f"{name}: {len(good)} matches, "
                         f"{len(inliers)} RANSAC inliers (80 strongest shown)"))

    # (c) Warp into the template frame; drop a band at the warped border
    aligned = cv2.warpPerspective(image, H, (tw, th))
    valid = cv2.warpPerspective(np.full(gray.shape, 255, np.uint8), H, (tw, th),
                                flags=cv2.INTER_NEAREST)
    valid = cv2.erode(valid, np.ones((15, 15), np.uint8)) > 0

    # Grayscale difference after exposure correction and light blurring
    aligned_gray = cv2.cvtColor(aligned, cv2.COLOR_BGR2GRAY).astype(np.float32)
    aligned_gray *= template_blur[valid].mean() / max(aligned_gray[valid].mean(), 1e-6)
    difference = np.abs(template_blur - cv2.GaussianBlur(aligned_gray, (0, 0), blur_sigma))
    difference[~valid] = 0
    difference = np.clip(difference, 0, 255).astype(np.uint8)

    # Large differences highlighted in red on the template
    changes = np.uint8(difference > change_threshold) * 255
    changes = cv2.morphologyEx(changes, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8)) > 0
    overlay = template.copy()
    overlay[changes] = (0, 0, 255)
    overlay = cv2.addWeighted(template, 0.4, overlay, 0.6, 0)

    rows.append((name, [(rgb(small(image)), "input"), (rgb(small(aligned)), "aligned"),
                        (small(difference), f"difference (0-{diff_display_max})"),
                        (rgb(small(overlay)), f"difference > {change_threshold}")]))
    summary.append((name, len(kp_image), len(good), len(inliers),
                    difference[valid].mean(), 100 * changes[valid].mean()))

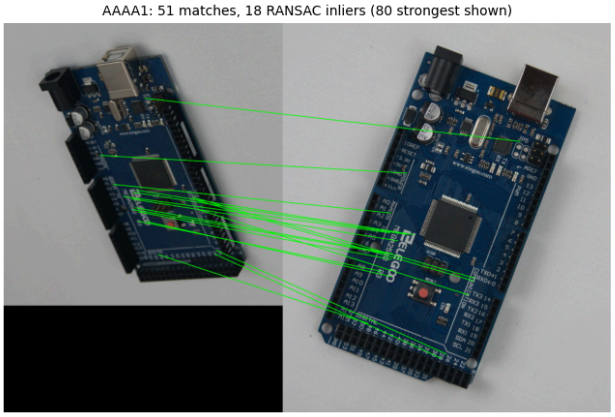
# (b) Matches for all images, two per row (one figure per row, so pages can break between rows)
for pair in (match_panels[i:i + 2] for i in range(0, len(match_panels), 2)):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4.4))
    for ax, (im, title) in zip(axes, pair):
        ax.imshow(im)
        ax.set_title(title, fontsize=10)
    for ax in axes:
        ax.axis("off")
    plt.tight_layout()
    plt.show()

# (d) Results for all images, one row each (difference images share one colour scale)
for name, panels in rows:
    fig, axes = plt.subplots(1, 4, figsize=(16, 5))
    for ax, (im, title) in zip(axes, panels):
        ax.imshow(im, cmap="inferno", vmin=0, vmax=diff_display_max)  # cmap: gray only
        ax.set_title(f"{name}: {title}")
        ax.axis("off")
    plt.tight_layout()
    plt.show()

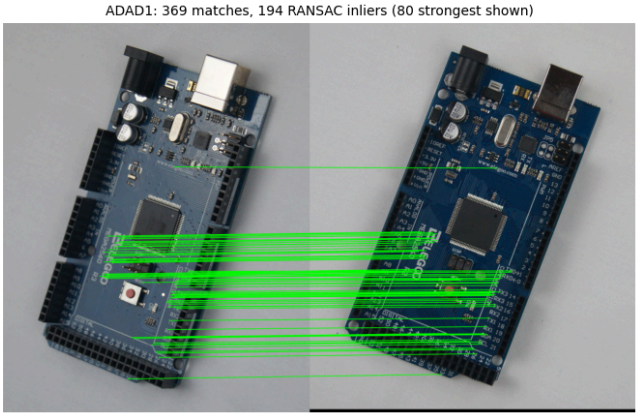
print(f"{'Image':<8}{'Keypoints':>10}{'Matches':>9}{'Inliers':>9}{'Ratio':>7}"
      f"{'Mean diff':>11}{'Changed %':>11}")
for name, kps, n_good, n_in, mean_diff, changed in summary:
    print(f"{name:<8}{kps:>10}{n_good:>9}{n_in:>9}{n_in / n_good:>7.3f}"
          f"{mean_diff:>11.2f}{changed:>11.2f}")
```

```
Template keypoints: 14272
```

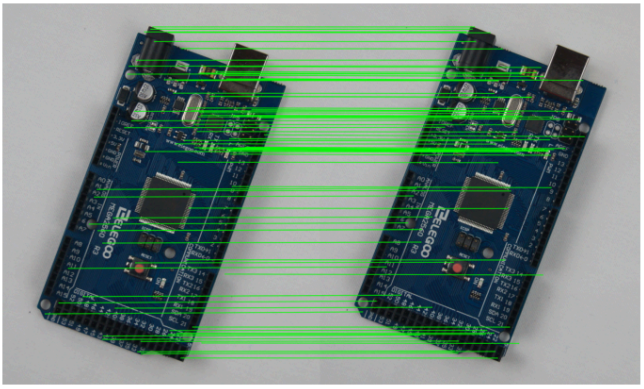
AAAA1: 51 matches, 18 RANSAC inliers (80 strongest shown)



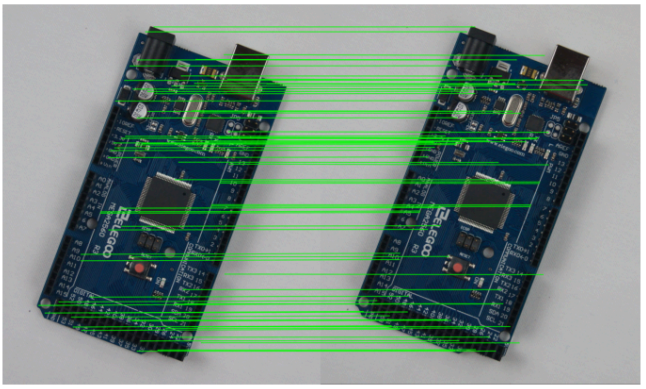
ADAD1: 369 matches, 194 RANSAC inliers (80 strongest shown)



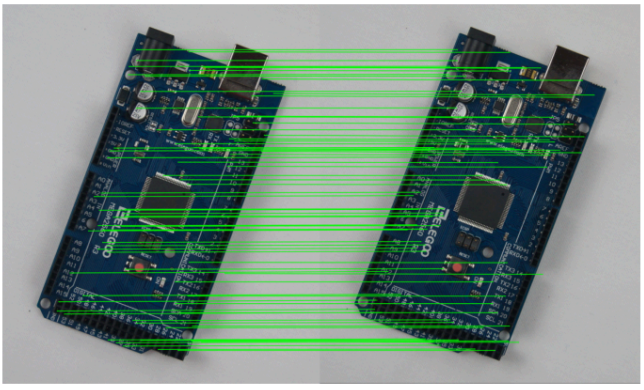
AECC2: 4322 matches, 3847 RANSAC inliers (80 strongest shown)



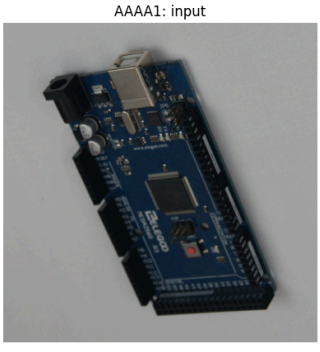
AECC3: 5629 matches, 5006 RANSAC inliers (80 strongest shown)



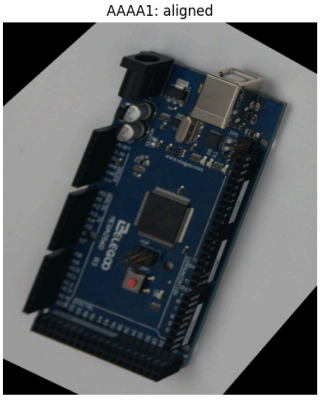
AECC4: 3830 matches, 3402 RANSAC inliers (80 strongest shown)



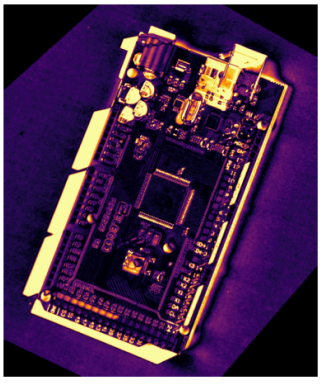
AAAA1: input



AAAA1: aligned



AAAA1: difference (0-80)



AAAA1: difference > 40

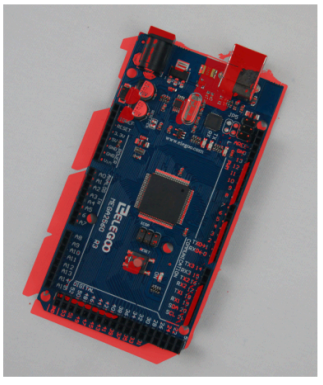




Image	Keypoints	Matches	Inliers	Ratio	Mean diff	Changed %
AAAA1	2414	51	18	0.353	22.65	12.38
ADAD1	8486	369	194	0.526	30.59	22.49
AECC2	12315	4322	3847	0.890	0.67	0.00
AECC3	15198	5629	5006	0.889	0.62	0.00
AECC4	11610	3830	3402	0.888	0.77	0.01

Difference computation. Both images are converted to grayscale, the aligned one is scaled to the template's mean brightness, and both are blurred ($\sigma = 2$) against noise and sub-pixel misalignment.

Difference images share one scale (0–80 gray levels); pixels differing by more than 40 are marked red after a 5×5 opening removes specks.

Results.

- **AECC2–4:** about 89% of matches are inliers and the mean difference is below 1 gray level; they appear to be AECC1 itself under a different warp.
- **AAAA1** (18 inliers) and **ADAD1** (194) are separate photographs. ADAD1's board is lighter blue with a white reset button, so its whole board differs; AAAA1, shot at a steep angle, is the least reliable alignment.

(e) Why the USB Type B port, capacitors and reset button show high difference.

- **Not on the board plane:** the homography is exact only for the PCB plane. The USB-B port, capacitors, DC jack, crystal and reset button stand millimetres above it, so a viewpoint change shifts them differently (parallax). Their sides, and those of the pin headers, are visible in one view but not another (the bright band along the headers in AAAA1).
- **Shiny surfaces:** the metal USB shell, capacitor tops and button reflect specularly, so their brightness depends on viewing and lighting directions, which global gain correction cannot fix.
- **Shadows:** tall parts cast shadows that move with the light and viewpoint.
- **Real differences:** e.g. ADAD1's reset button colour.